# C# Interview Questions and Answers

## Table of Contents



---

## Basic Level Questions

### 1. What is C# and what are its key features?

**Answer:** C# (C-Sharp) is a modern, object-oriented programming language developed by Microsoft as part of the .NET framework.

**Key Features:**

- Object-oriented programming (OOP)
- Type-safe language
- Automatic garbage collection
- Rich library support
- Cross-platform development with .NET Core/. NET 5+
- LINQ (Language Integrated Query)
- Async/await for asynchronous programming
- Strong type checking

### 2. What is the difference between value types and reference types?

**Answer:**

**Value Types:**

- Stored on the stack
- Contain actual data
- Examples: int, float, double, struct, enum
- When assigned, a copy of the value is created
- Derived from System.ValueType

```
int a = 10;
int b = a;  // 'b' gets a copy of value 10
b = 20;     // 'a' is still 10
```

**Reference Types:**

- Stored on the heap
- Contain reference/address to actual data
- Examples: class, interface, delegate, string, array
- When assigned, both variables point to same object
- Derived from System. Object

```
class Person { public string Name; }
Person p1 = new Person { Name = "John" };
Person p2 = p1;  // Both point to same object
p2.Name = "Jane"; // p1.Name is also "Jane"
```

### 3. What is boxing and unboxing?

**Answer:**

**Boxing:** Converting a value type to reference type (object).

```
int num = 123;
object obj = num;  // Boxing
```

**Unboxing:** Converting a reference type back to value type.

```
object obj = 123;
int num = (int)obj;  // Unboxing
```

**Performance Note:** Boxing/unboxing creates overhead and should be avoided when possible. Use generics instead.

## 4. What are the four pillars of OOP?

**Answer:**

1. **Encapsulation:** Hiding internal details and showing only necessary information

```
public class Account
{
    private decimal balance;  // Hidden

    public void Deposit(decimal amount)  // Public interface
    {
        if (amount > 0)
            balance += amount;
    }
}
```

2. **Inheritance:** Ability to create new classes from existing ones

```
public class Animal { }
public class Dog : Animal { }
```

3. **Polymorphism:** Ability to take multiple forms

```
public virtual void MakeSound() { }  // Can be overridden
```

4. **Abstraction:** Hiding complex implementation details

```
public abstract class Shape
{
    public abstract double GetArea();
}
```

## 5. What is the difference between abstract class and interface?

**Answer:**

| Feature | Abstract Class | Interface |
|---|---|---|
| Multiple Inheritance | Not supported | Supported |
| Access Modifiers | Can have any | All members are public by default |
| Implementation | Can have implementation | No implementation (before C# 8.0) |
| Fields | Can have fields | Cannot have fields |
| Constructors | Can have constructors | Cannot have constructors |
| When to use | "is-a" relationship | "can-do" relationship |

```
// Abstract Class
public abstract class Vehicle
{
    protected string brand;
    public abstract void Start();

    public void ShowBrand()  // Concrete method
    {
        Console.WriteLine(brand);
    }
}

// Interface
public interface IDriveable
{
    void Drive();
    void Stop();
}
```

---

## Intermediate Level Questions

### 6. What is the difference between `const` and `readonly`?

**Answer:**

| Feature | const | readonly |
|---|---|---|
| Assignment | At compile time | At compile time or runtime |
| Declaration | Must be initialized at declaration | Can be initialized in constructor |
| Scope | Implicitly static | Can be instance or static |
| Type | Only value types and strings | Any type |

```
public class Example
{
    public const int MaxValue = 100;  // Compile-time constant
    public readonly int MinValue;     // Runtime constant

    public Example(int min)
    {
        MinValue = min;  // Can initialize in constructor
    }
}
```

### 7. Explain the difference between `String` and `StringBuilder`.

**Answer:**

**String:**

- Immutable (cannot be changed after creation)
- Each modification creates a new string object
- Better for fixed strings or few modifications

**StringBuilder:**

- Mutable (can be modified)
- More efficient for multiple string operations
- Better for string concatenation in loops

```
// String - Creates multiple objects
string str = "Hello";
str += " World";  // Creates new string object
str += "!";       // Creates another new object

// StringBuilder - Modifies same object
StringBuilder sb = new StringBuilder("Hello");
sb.Append(" World");  // Modifies existing object
sb.Append("!");       // Still same object
```

## 8. What are delegates and what are the different types?

**Answer:**

A delegate is a type-safe function pointer that holds reference to methods.

**Types of Delegates:**

1. **Single-cast Delegate:** Points to single method

```
public delegate void MyDelegate(string message);

MyDelegate del = ShowMessage;
del("Hello");

void ShowMessage(string msg) => Console.WriteLine(msg);
```

2. **Multicast Delegate:** Points to multiple methods

```
MyDelegate del = Method1;
del += Method2;  // Adds another method
del("Test");     // Calls both methods
```

3. **Built-in Delegates:**

- **Action:** No return value

```
Action<string> action = msg => Console.WriteLine(msg);
```

- **Func:** Has return value

```
Func<int, int, int> add = (a, b) => a + b;
```

- **Predicate:** Returns bool

```
Predicate<int> isEven = num => num % 2 == 0;
```

## 9. What is the difference between `IEnumerable` and `IQueryable`?

**Answer:**

| Feature | IEnumerable | IQueryable |
|---|---|---|
| Namespace | System.Collections | System.Linq |
| Execution | In-memory | On data source |
| Best for | In-memory collections | Remote data (DB queries) |
| Deferred Execution | Yes | Yes |
| Query Execution | Client-side | Server-side |

```
// IEnumerable - All data loaded to memory first
IEnumerable<Product> products = dbContext.Products. ToList();
var filtered = products.Where(p => p.Price > 100);  // Filtered in memory

// IQueryable - Filter applied at database
IQueryable<Product> products = dbContext.Products;
var filtered = products.Where(p => p.Price > 100);  // Filtered in SQL
```

### 10. Explain async/await in C#.

**Answer:**

Async/await enables asynchronous programming, allowing code to run without blocking the main thread.

```
public async Task<string> GetDataAsync()
{
    // Simulating async operation
    await Task. Delay(1000);
    return "Data loaded";
}

// Usage
public async Task ProcessDataAsync()
{
    Console.WriteLine("Starting.. .");

    string result = await GetDataAsync();  // Doesn't block

    Console.WriteLine(result);
    Console.WriteLine("Completed");
}
```

**Key Points:**

- `async` keyword marks method as asynchronous
- `await` pauses execution until task completes
- Returns `Task` or `Task<T>`
- Improves application responsiveness
- Doesn't create new threads automatically

---

## Advanced Level Questions

### 11. What are extension methods and how do you create them?

**Answer:**

Extension methods allow adding new methods to existing types without modifying them.

**Requirements:**

- Must be static method in static class
- First parameter uses `this` modifier

```
public static class StringExtensions
{
    public static bool IsValidEmail(this string email)
    {
        return email.Contains("@") && email.Contains(".");
    }

    public static string Reverse(this string str)
    {
        char[] charArray = str.ToCharArray();
        Array.Reverse(charArray);
        return new string(charArray);
    }
}

// Usage
string email = "test@example.com";
bool isValid = email.IsValidEmail();  // true

string word = "Hello";
string reversed = word. Reverse();  // "olleH"
```

## 12. Explain LINQ and provide examples of different LINQ operations.

**Answer:**

LINQ (Language Integrated Query) provides a consistent way to query data from different sources.

```
List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// Query Syntax
var evenQuery = from num in numbers
                where num % 2 == 0
                select num;

// Method Syntax
var evenMethod = numbers.Where(n => n % 2 == 0);

// Common LINQ Operations
var filtered = numbers.Where(n => n > 5);           // Filtering
var squared = numbers.Select(n => n * n);           // Projection
var sorted = numbers.OrderBy(n => n);               // Sorting
var first = numbers.First(n => n > 5);              // 6
var sum = numbers.Sum();                            // 55
var average = numbers.Average();                    // 5.5
var grouped = numbers.GroupBy(n => n % 2 == 0);    // Grouping

// Complex example with objects
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
    public int Grade { get; set; }
}

List<Student> students = new List<Student>
{
    new Student { Name = "Alice", Age = 20, Grade = 85 },
    new Student { Name = "Bob", Age = 22, Grade = 92 },
    new Student { Name = "Charlie", Age = 21, Grade = 78 }
};

var topStudents = students
    .Where(s => s.Grade > 80)
    .OrderByDescending(s => s.Grade)
    .Select(s => new { s.Name, s.Grade });
```

## 13. What are generics and why are they useful?

**Answer:**

Generics allow creating type-safe classes, methods, and interfaces without specifying the exact type until instantiation.

**Benefits:**

- Type safety at compile time
- Code reusability
- Better performance (no boxing/unboxing)
- IntelliSense support

```
// Generic Class
public class Repository<T> where T : class
{
    private List<T> items = new List<T>();

    public void Add(T item)
    {
        items.Add(item);
    }

    public T Get(int index)
    {
        return items[index];
    }

    public IEnumerable<T> GetAll()
    {
        return items;
    }
}

// Generic Method
public T Max<T>(T a, T b) where T : IComparable<T>
{
    return a.CompareTo(b) > 0 ? a : b;
}

// Usage
Repository<Product> productRepo = new Repository<Product>();
productRepo.Add(new Product { Id = 1, Name = "Laptop" });

int maxNum = Max(10, 20);  // Type inference
```

## 14. Explain the different types of design patterns with examples.

**Answer:**

1. **Singleton Pattern:** Ensures only one instance of a class exists

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> instance =
        new Lazy<Singleton>(() => new Singleton());

    private Singleton() { }

    public static Singleton Instance => instance.Value;
}
```

2. **Factory Pattern:** Creates objects without specifying exact class

```
public interface IShape
{
    void Draw();
}

public class Circle : IShape
{
    public void Draw() => Console.WriteLine("Circle");
}

public class Square : IShape
{
    public void Draw() => Console.WriteLine("Square");
}

public class ShapeFactory
{
    public IShape GetShape(string type)
    {
        return type switch
        {
            "circle" => new Circle(),
            "square" => new Square(),
            _ => throw new ArgumentException("Invalid shape")
        };
    }
}
```

3. **Repository Pattern:** Abstracts data access logic

```
public interface IRepository<T>
{
    T GetById(int id);
    IEnumerable<T> GetAll();
    void Add(T entity);
    void Update(T entity);
    void Delete(int id);
}

public class ProductRepository : IRepository<Product>
{
    private readonly DbContext _context;

    public ProductRepository(DbContext context)
    {
        _context = context;
    }

    public Product GetById(int id) => _context.Products.Find(id);
    // ... other implementations
}
```

## 15. What is dependency injection and why is it important?

**Answer:**

Dependency Injection (DI) is a design pattern where objects receive dependencies from external sources rather than creating them.

**Benefits:**

- Loose coupling
- Better testability
- Easier maintenance
- Follows SOLID principles

```
// Without DI - Tight coupling
public class OrderService
{
    private EmailService emailService = new EmailService();  // Bad

    public void PlaceOrder(Order order)
    {
        // Process order
        emailService.SendConfirmation(order);
    }
}

// With DI - Loose coupling
public interface IEmailService
{
    void SendConfirmation(Order order);
}

public class EmailService : IEmailService
{
    public void SendConfirmation(Order order)
    {
        // Send email
    }
}

public class OrderService
{
    private readonly IEmailService _emailService;

    // Constructor injection
    public OrderService(IEmailService emailService)
    {
        _emailService = emailService;
    }

    public void PlaceOrder(Order order)
    {
        // Process order
        _emailService. SendConfirmation(order);
    }
}

// Registration in ASP.NET Core
public void ConfigureServices(IServiceCollection services)
{
    services.AddScoped<IEmailService, EmailService>();
    services.AddScoped<OrderService>();
}
```

## 16. Explain Exception Handling best practices in C#.

**Answer:**

```
public class ExceptionHandlingExample
{
    public void BestPractices()
    {
        // 1. Catch specific exceptions first
        try
        {
            // Some operation
        }
        catch (FileNotFoundException ex)
        {
            // Handle specific exception
            Console.WriteLine($"File not found: {ex.Message}");
        }
        catch (IOException ex)
        {
            // Handle broader exception
            Console.WriteLine($"IO error: {ex.Message}");
        }
        catch (Exception ex)
        {
            // Handle general exception last
            Console.WriteLine($"Error: {ex.Message}");
            throw;  // Re-throw to preserve stack trace
        }
        finally
        {
            // Cleanup code (always executes)
        }
    }

    // 2. Create custom exceptions
    public class InvalidOrderException : Exception
    {
        public InvalidOrderException(string message) : base(message) { }
    }

    // 3. Use using statement for IDisposable
    public void ReadFile(string path)
    {
        using (StreamReader reader = new StreamReader(path))
        {
            // File automatically closed even if exception occurs
            string content = reader.ReadToEnd();
        }
    }

    // 4. Don't catch exceptions you can't handle
    // 5. Log exceptions properly
    // 6. Use exception filters (C# 6.0+)
    public void ExceptionFilter()
    {
        try
        {
            // Some operation
        }
        catch (Exception ex) when (ex.Message.Contains("timeout"))
        {
            // Only catch if condition is true
        }
    }
}
```

### 17. What is the difference between `==` and `.Equals()`?

**Answer:**

```
// == operator
// - For value types:  compares values
// - For reference types: compares references (unless overridden)

// .Equals() method
// - Can be overridden to provide custom comparison logic
// - Default behavior same as == for reference types

int a = 5, b = 5;
Console. WriteLine(a == b);        // True (value comparison)
Console.WriteLine(a. Equals(b));   // True (value comparison)

string s1 = "hello";
string s2 = "hello";
Console.WriteLine(s1 == s2);      // True (string overrides ==)
Console.WriteLine(s1.Equals(s2)); // True

object obj1 = new object();
object obj2 = obj1;
object obj3 = new object();
Console.WriteLine(obj1 == obj2);      // True (same reference)
Console.WriteLine(obj1 == obj3);      // False (different reference)
Console.WriteLine(obj1.Equals(obj3)); // False

// Custom class example
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public override bool Equals(object obj)
    {
        if (obj is Person other)
            return Name == other.Name && Age == other.Age;
        return false;
    }

    public override int GetHashCode()
    {
        return HashCode.Combine(Name, Age);
    }
}
```

## 18. What are Records in C# 9.0 and when should you use them?

**Answer:**

Records are reference types designed for immutable data with value-based equality.

```
// Record declaration
public record Person(string FirstName, string LastName, int Age);

// Usage
var person1 = new Person("John", "Doe", 30);
var person2 = new Person("John", "Doe", 30);

Console.WriteLine(person1 == person2);  // True (value equality)

// with expression (non-destructive mutation)
var person3 = person1 with { Age = 31 };

// Traditional class equivalent
public class PersonClass
{
    public string FirstName { get; init; }
    public string LastName { get; init; }
    public int Age { get; init; }

    // Would need to override Equals, GetHashCode, ToString
}

// When to use Records:
// - DTOs (Data Transfer Objects)
// - Immutable data models
// - Value objects
// - When you need value-based equality
```

---

## Bonus: Practical Coding Questions

### 19. Reverse a string without using built-in methods

```
public string ReverseString(string input)
{
    if (string.IsNullOrEmpty(input))
        return input;

    char[] charArray = input.ToCharArray();
    int left = 0;
    int right = charArray.Length - 1;

    while (left < right)
    {
        char temp = charArray[left];
        charArray[left] = charArray[right];
        charArray[right] = temp;
        left++;
        right--;
    }

    return new string(charArray);
}
```

### 20. Find duplicate elements in an array

```
public List<int> FindDuplicates(int[] array)
{
    Dictionary<int, int> frequency = new Dictionary<int, int>();
    List<int> duplicates = new List<int>();

    foreach (int num in array)
    {
        if (frequency.ContainsKey(num))
        {
            frequency[num]++;
            if (frequency[num] == 2)  // Add only once
                duplicates.Add(num);
        }
        else
        {
            frequency[num] = 1;
        }
    }

    return duplicates;
}

// Using LINQ
public IEnumerable<int> FindDuplicatesLinq(int[] array)
{
    return array.GroupBy(x => x)
                .Where(g => g.Count() > 1)
                .Select(g => g.Key);
}
```

---

## Additional Resources

- Official C# Documentation: https://docs.microsoft.com/en-us/dotnet/csharp/
- C# Programming Guide
- Design Patterns in C#
- SOLID Principles
- Clean Code practices